Table 6.4 Common HTTP request headers

Request header	Description
Authorization	The credentials for making an authenticated request. If the server uses OAuth2 to authorize users, the token would be set in this header.
Content-Type	The MIME type of the request body. The MIME type indicates the data type of the body. If the body were JSON, the MIME type would be `application/json`.
If-Match	The ETag value of the corresponding resource. Used during updates to ensure that the update doesn't conflict with a different update. Also used for requests with the Range header to make sure the new part matched the previously downloaded parts.
If-Modified-Since	Similar to the If-None-Match header, but with a date instead of ETag. The server only returns a response body if the resource has been modified since the date in this header. Otherwise, the server returns an HTTP 304 Not Modified status.
If-None-Match	The ETag value of the corresponding resource. Used when retrieving a resource. If the resource's ETag matches the request ETag, the resource hasn't changed since the previous retrieval, and the server returns an HTTP 304 Not Modified status (without a response body).
If-Unmodifed-Since	Similar to If-Match. Used during updates to ensure that the update doesn't conflict with a different update and to check if a Range request makes sense. Uses date instead of ETag.
Range	Specifies a range of bytes to download from the server (rather than the entire response). Used during resumable downloads to specify where to start the downloading.
X-Requested-With	Indicates where an AJAX request originates. Set by many JavaScript clients (although JQuery disables this header when making CORS requests).

Some of these headers are tied to certain features. For example, the If-Match and If-None-Match headers are tied to ETag support, so those headers need only be allowed if your server supports ETags. Table 6.4 is a good place to start if you're trying to decide which HTTP headers to support.

It's not easy to configure HTTP headers because there can literally be an infinite number of custom response headers. As a server administrator, you don't want to continually update your server configuration every time a client sends a new header. If you find yourself in this situation, the next section shares code to allow all request headers through.

ALLOWING ALL REQUEST HEADERS

As a server administrator, it can be overwhelming to manage all the headers a client may send. You may not even care about which headers a client includes on the request. In these cases, it would be helpful if the Access-Control-Allow-Headers header supported a `*` value that let all request headers through. The CORS spec doesn't define a `*` value for the Access-Control-Allow-Headers header, but you can mimic this behavior through the code in the following listing.